

Universidad Politécnica de Chiapas



Nombres de los alumnos:

Escobar Nuricumbo Heber Alexander

González Abarca Edy Jordan

Ramírez Trujillo Karolina Guadalupe

Matricula:

243691

243842

243733

Materia:

Base de Datos Avanzadas

Docente:

Ing. Ramsés Alejandro Camas Najera

Actividad:

Examen: Diseño de sistemas y Base de Datos

05 de Febrero del 2026.

Suchiapa, Chiapas.

1. Restricciones identificadas:

1. UNIQUE (4 restricciones)			
#	Tabla	Campo	Justificación
1	PROVEEDOR	rfc	El RFC es un identificador fiscal único por persona moral/física. Evita duplicados de proveedores
2	CATEGORIA	nombre	Evita categorías duplicadas (ej: no puede haber dos "Lácteos")
3	VEHICULO	placas	Cada unidad de la flota debe ser única e irrepetible en el sistema
4	CONDUCTOR	licencia	La licencia de conducir es un identificador único por persona (ej: #LIC-9876)

2. CHECK (15 restricciones)				
#	Tabla	Campo	Restricción SQL	Justificación
5	MOVIMIENTO_INVENTARIO	tipo	tipo IN ('ENTRADA', 'SALIDA', 'AJUSTE')	Garantiza que solo se registren tipos válidos de movimiento. Escalable

				para nuevos tipos
6	VEHICULO	estado	estado IN ('Disponible', 'En ruta', 'Mantenimiento')	CRÍTICO: Previene asignar vehículos en mantenimiento a rutas
7	COMERCIO	zona	zona IN ('Fronteriza', 'Istmo', 'Centro', 'Tabasco')	Valida zonas geográficas del caso. Base para reportes por zona
8	ruta	estado	estado IN ('Planeada', 'En ruta', 'Completada')	Flujo lógico del ciclo de vida de una ruta
9	PEDIDO	estado	estado IN ('Pendiente', 'Confirmado', 'Entregado', 'Entregado con incidencia')	Estados del pedido según el caso. Distingue entregas exitosas de problemáticas
10	RUTA_PEDIDO	estado_entrega	estado_entrega IN ('Pendiente', 'Entregado', 'Con incidencia')	Tracking granular de cada entrega individual
11	PRODUCTO	precio_compra	precio_compra > 0	No existen productos gratuitos de proveedores
12	PRODUCTO	precio_venta	precio_venta > 0	No se pueden regalar productos
13	PRODUCTO	stock_actual	stock_actual >= 0	Evita inventarios negativos. El stock mínimo

				es cero
14	DETALLE_PEDIDO	cantidad	cantidad > 0	No se pueden pedir cero o productos negativos
15	DETALLE_PEDIDO	precio_unitario	precio_unitario > 0	Precio al momento del pedido debe ser positivo
16	DETALLE_PEDIDO	ubtotal	subtotal > 0	Subtotal calculado debe ser positivo
17	VEHICULO	capacidad_kg	capacidad_kg > 0	Un vehículo debe tener capacidad de carga
18	MOVIMIENTO_INVENTARIO	cantidad	cantidad > 0	Las cantidades movidas deben ser positivas
19	ruta_pedido	orden_entrega	orden_entrega > 0	Secuencia de entregas debe ser positiva

3. NOT NULL (21 restricciones)			
#	Tabla	Campo	Justificación
20	PROVEEDOR	rfc	RFC es obligatorio para identificación fiscal
21	PROVEEDOR	nombre	Un proveedor debe tener razón social
22	CATEGORIA	nombre	Una categoría debe tener

			identificador
23	ALMACEN	nombre	Nombre del almacén es obligatorio
24	ALMACEN	ciudad	Ubicación geográfica obligatoria
25	PRODUCTO	nombre	Nombre del producto obligatorio
26	PRODUCTO	unidad_medida	Debe especificar kg/litros/piezas
27	PRODUCTO	stock_actual	stock_actual
28	PRODUCTO	id_categoria	Integridad referencial: Todo producto debe pertenecer a una categoría
29	PRODUCTO	id_proveedor	Integridad referencial: Todo producto debe estar ligado a un proveedor (Pregunta 2)
30	MOVIMIENTO_IN VENTARIO	tipo	Tipo de movimiento obligatorio
31	MOVIMIENTO_IN VENTARIO	fecha	Registro temporal obligatorio
32	COMERCIO	zona	Zona obligatoria para reportes geográficos
33	PEDIDO	fecha	Fecha del pedido obligatoria
34	PEDIDO	estado	Estado del pedido obligatorio
35	PEDIDO	id_comercio	Integridad

			referencial: Un pedido no puede ser "anónimo", debe pertenecer a una tienda
36	VEHICULO	placas	Placas obligatorias para identificación
37	CONDUCTOR	licencia	Licencia obligatoria para operar
38	RUTA	id_vehiculo	Toda ruta debe tener vehículo asignado
39	RUTA	id_conductor	Toda ruta debe tener conductor asignado
40	RUTA_PEDIDO	estado_entrega	Estado de entrega obligatorio para tracking

4. FOREIGN KEY - Integridad Referencial (12 restricciones)					
#	Tabla	Campo	Referencia	Acción	Justificación
41	PRODUCTO	id_categoria	CATEGORIA(id_categoria)	ON DELETE RESTRICT	Un producto debe pertenecer a una categoría válida. No se puede eliminar categoría con productos
42	PRODUCTO	id_proveedor	PROVEEDOR(id_proveedor)	ON DELETE	Un producto

			edor)	RESTRICT	debe tener un proveedor existente. No se puede eliminar proveedor activo
43	MOVIMIENTO_INVENTARIO	id_producto	PRODUCTO(id_producto)	ON DELETE RESTRICT	Todo movimiento debe estar asociado a un producto existente
44	MOVIMIENTO_INVENTARIO	id_almacen	ALMACEN(id_almacen)	ON DELETE RESTRICT	Todo movimiento ocurre en un almacén específico.
45	MOVIMIENTO_INVENTARIO	id_pedido	PEDIDO(id_pedido)	ON DELETE SET NULL	Asocia salidas con pedidos. NULLABLE para entradas y ajustes.
46	PEDIDO	id_comercio	COMERCIO(id_comercio)	ON DELETE RESTRICT	Todo pedido debe pertenecer a un comercio existente
47	DETALLE_PEDIDO	id_pedido	PEDIDO(id_pedido)	ON DELETE CASCADE	Entidad débil: si se elimina el pedido, se eliminan sus detalles

48	DETALLE_PEDIDO	id_producto	PRODUCTO(id_producto)	ON DELETE RESTRICT	Cada línea del pedido especifica un producto. Permite múltiples proveedores.
49	RUTA	id_vehiculo	VEHICULO(id_vehiculo)	ON DELETE RESTRICT	Toda ruta debe tener un vehículo asignado
50	RUTA	id_conductor	CONDUCTOR(id_conductor)	ON DELETE RESTRICT	Toda ruta debe tener un conductor asignado
51	RUTA_PEDIDO	id_ruta	RUTA(id_ruta)	ON DELETE CASCADE	Entidad débil: si se elimina la ruta, se eliminan sus asignaciones de pedidos (N-N)
52	RUTA_PEDIDO	id_pedido	PEDIDO(id_pedido)	ON DELETE CASCADE	Entidad débil: indica qué pedidos van en cada ruta (N-N)

2.- Queries SQL

2.1.- Precio total asignado por Ruta vs Capacidad del Vehículo

Técnicas: JOIN (múltiple), SUM (Agregación), GROUP BY.

```
SELECT
    r.id_ruta,
    v.placas,
    v.capacidad_kg,
    SUM(dp.cantidad) AS peso_total_cargado,
    (v.capacidad_kg - SUM(dp.cantidad)) AS margen_disponible
FROM RUTA r
JOIN VEHICULO v ON r.id_vehiculo = v.id_vehiculo
JOIN RUTA_PEDIDO rp ON r.id_ruta = rp.id_ruta
JOIN DETALLE_PEDIDO dp ON rp.id_pedido = dp.id_pedido
GROUP BY r.id_ruta, v.placas, v.capacidad_kg;
```

2.2.- Valor total del Inventario por Categoría

Técnicas: JOIN, SUM, COALESCE. Calcula cuánto dinero hay invertido en el almacén. Usamos COALESCE para asegurar que si el stock es nulo, se trate como 0.

```
SELECT
    c.nombre AS categoria,
    SUM(COALESCE(p.stock_actual, 0) * p.precio_compra) AS valor_total_inventario
FROM CATEGORIA c
JOIN PRODUCTO p ON c.id_categoria = p.id_categoria
GROUP BY c.nombre;
```

2.3.- Movimientos de Entrada del último mes por Proveedor

Técnicas: JOIN, COUNT, GROUP BY, HAVING. Filtra solo a los proveedores que han enviado una cantidad significativa de productos (más de 1 movimiento).

```
SELECT
    prov.nombre AS proveedor,
    COUNT(m.id_movimiento) AS total_entradas,
    SUM(m.cantidad) AS unidades_recibidas
FROM PROVEEDOR prov
JOIN PRODUCTO prod ON prov.id_proveedor = prod.id_proveedor
JOIN MOVIMIENTO_INVENTARIO m ON prod.id_producto = m.id_producto
WHERE m.tipo = 'ENTRADA'
AND m.fecha >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
```

```
GROUP BY prov.nombre  
HAVING COUNT(m.id_movimiento) > 0;
```

2.4.- Reporte de Estatus de Entregas (Caso Don Pepe y La Esquina)

Técnicas: JOIN, CASE, COALESCE. Muestra los pedidos y traduce el estado de la entrega a un mensaje legible, manejando las incidencias.

```
SELECT  
    p.id_pedido,  
    com.nombre AS comercio,  
    com.zona,  
    rp.estado_entrega,  
    CASE  
        WHEN rp.estado_entrega = 'Entregado' THEN 'Éxito Total'  
        WHEN rp.estado_entrega = 'Entregado con incidencia' THEN  
CONCAT('Problema: ', COALESCE(rp.incidencia, 'No especificada'))  
        ELSE 'En proceso'  
    END AS reporte_estatus  
FROM PEDIDO p  
JOIN COMERCIO com ON p.id_comercio = com.id_comercio  
JOIN RUTA_PEDIDO rp ON p.id_pedido = rp.id_pedido;
```

3.- Reportes Implementados (VIEWS)

3.1.- Resumen de Pedidos por Zona Geográfica

Este reporte analiza el desempeño logístico por zona. Es vital para identificar dónde hay más problemas de entrega.

```
CREATE VIEW vista_resumen_zonas AS  
SELECT  
    c.zona,  
    COUNT(p.id_pedido) AS total_pedidos,  
    SUM(CASE WHEN p.estado = 'Entregado' THEN 1 ELSE 0 END) AS  
entregas_exitosas,  
    SUM(CASE WHEN p.estado = 'Entregado con incidencia' THEN 1 ELSE 0 END)  
AS entregas_con_incidencia,  
    ROUND((SUM(CASE WHEN p.estado = 'Entregado' THEN 1 ELSE 0 END) *  
100.0) / COUNT(p.id_pedido), 2) AS tasa_exito_porcentaje  
FROM COMERCIO c  
JOIN PEDIDO p ON c.id_comercio = p.id_comercio
```

```
GROUP BY c.zona
HAVING COUNT(p.id_pedido) >= 1; -- Solo zonas con actividad
```

3.2.- Productividad de Conductores

Evalúa el rendimiento de los conductores basándose en rutas y pedidos completados. Incluye la **clasificación de rendimiento** solicitada.

```
CREATE VIEW vista_productividad_conductores AS
SELECT
    cond.nombre AS conductor,
    COUNT(DISTINCT r.id_ruta) AS rutas_asignadas,
    COUNT(rp.id_pedido) AS pedidos_entregados,
    ROUND(COUNT(rp.id_pedido) * 1.0 / COUNT(DISTINCT r.id_ruta), 2) AS
    promedio_por_ruta,
    CASE
        WHEN COUNT(rp.id_pedido) > 15 THEN 'Excelente (Alto Rendimiento)'
        WHEN COUNT(rp.id_pedido) BETWEEN 6 AND 15 THEN 'Estándar (Cumple)'
        ELSE 'Bajo (Requiere Revisión)'
    END AS clasificacion_rendimiento
FROM CONDUCTOR cond
JOIN RUTA r ON cond.id_conductor = r.id_conductor
LEFT JOIN RUTA_PEDIDO rp ON r.id_ruta = rp.id_ruta
GROUP BY cond.id_conductor, cond.nombre;
```

3.3.- Inventario Crítico (Sin error de división por cero)

Identifica productos que se agotarán en menos de 15 días. Usamos NULLIF y COALESCE para manejar productos como la "Salsa artesanal" que no tiene ventas.

```
CREATE VIEW vista_inventario_critico AS
SELECT
    p.nombre AS producto,
    p.stock_actual,
    COALESCE(SUM(dp.cantidad), 0) AS ventas_ultimos_30_dias,
    CASE
        WHEN COALESCE(SUM(dp.cantidad), 0) = 0 THEN 'Sin ventas recientes'
        WHEN p.stock_actual < (SUM(dp.cantidad) / 30.0) * 15 THEN
        'REABASTECIMIENTO URGENTE'
        ELSE 'Stock Seguro'
    END AS estatus_critico
FROM PRODUCTO p
LEFT JOIN DETALLE_PEDIDO dp ON p.id_producto = dp.id_producto
GROUP BY p.id_producto, p.nombre, p.stock_actual;
```

3.4.- Rentabilidad por Proveedor

Compara costos de adquisición vs. ingresos por ventas. Fundamental para la toma de decisiones financieras.

```
CREATE VIEW vista_rentabilidad_proveedor AS
SELECT
    prov.nombre AS proveedor,
    SUM(dp.cantidad * p.precio_compra) AS inversion_total,
    SUM(dp.cantidad * dp.precio_unitario) AS venta_total,
    SUM(dp.cantidad * (dp.precio_unitario - p.precio_compra)) AS margen_ganancia,
    ROUND((SUM(dp.cantidad * (dp.precio_unitario - p.precio_compra)) /
    NULLIF(SUM(dp.cantidad * dp.precio_unitario), 0)) * 100, 2) AS porcentaje_margen
FROM PROVEEDOR prov
JOIN PRODUCTO p ON prov.id_proveedor = p.id_proveedor
JOIN DETALLE_PEDIDO dp ON p.id_producto = dp.id_producto
GROUP BY prov.id_proveedor, prov.nombre;
```

3.5. Utilización de la Flota

Muestra qué vehículos están trabajando más y detecta los que están en mantenimiento o subutilizados.

```
CREATE VIEW vista_utilizacion_flota AS
SELECT
    v.placas,
    v.capacidad_kg,
    v.estado AS estatus_actual,
    COUNT(r.id_ruta) AS rutas_en_el_mes,
    CASE
        WHEN v.estado = 'Mantenimiento' THEN 'Fuera de Servicio'
        WHEN COUNT(r.id_ruta) > 20 THEN 'Sobrecargado'
        WHEN COUNT(r.id_ruta) BETWEEN 10 AND 20 THEN 'Punto Óptimo'
        ELSE 'Subutilizado'
    END AS diagnostico_uso
FROM VEHICULO v
LEFT JOIN RUTA r ON v.id_vehiculo = r.id_vehiculo
```

GROUP BY v.id_vehiculo, v.placas, v.capacidad_kg, v.estado;

4.- Justificación

¿Por qué escogieron este caso?

Elegimos el **Caso 3-A: Logística de Distribución Regional** por tres razones fundamentales:

- **Desafío de Integridad de Datos:** A diferencia de otros casos, la logística exige un control estricto sobre los estados de los activos (vehículos en mantenimiento) y la trazabilidad de los productos. Esto nos permitió aplicar restricciones (CHECKS y TRIGGERS) mucho más interesantes y realistas.
- **Complejidad en la Relación de Negocio:** El flujo de Proveedor. Almacén. Ruta, Comercio presenta una jerarquía clara pero compleja. El hecho de que un pedido pueda tener productos de múltiples proveedores y que una ruta pueda consolidar varios pedidos nos obligó a diseñar un modelo altamente relacional.
- **Análisis de Datos Críticos:** Los requerimientos de reportes (como la división entre cero en productos sin ventas o el cálculo de rentabilidad) son problemas clásicos de ingeniería de software que queríamos resolver elegantemente mediante el uso de VIEWS y lógica de SQL.

¿Por qué lo diseñamos así?

Nuestro diseño se basa en tres pilares arquitectónicos: Normalización, Trazabilidad y Escalabilidad.

A. Uso de Tablas Puente (Relaciones N-N)

Implementamos dos relaciones Muchos-a-Muchos fundamentales:

DETALLE_PEDIDO: No solo rompe la relación entre Pedido y Producto, sino que actúa como una entidad débil que registra el precio histórico al momento de la venta, protegiendo al sistema de cambios futuros en los precios del catálogo.

RUTA_PEDIDO: Esta es nuestra tabla logística clave. Decidimos colocar aquí la firma_digital y las incidencias porque estas no pertenecen al pedido en sí, sino al evento de la entrega dentro de una ruta específica.

B. Centralización del Inventario

En lugar de tener tablas separadas para entradas y salidas, optamos por una tabla única de MOVIMIENTO_INVENTARIO.

¿Por qué? Porque facilita la auditoría. Si mañana el negocio requiere un nuevo tipo de movimiento (como "Mermas" o "Devoluciones"), solo añadimos un valor al CHECK del campo tipo, sin alterar la estructura de la base de datos (cumpliendo con el principio de Abierto/Cerrado).

C. Control de Estado y Reglas de Negocio

Diseñamos el sistema para que sea "consciente" del estado de sus recursos. Al incluir el campo estado en la tabla VEHICULO, permitimos que la capa de aplicación o un Trigger bloquee la creación de rutas con unidades en mantenimiento, garantizando que la base de datos sea el primer filtro de seguridad del negocio.

D. Resiliencia en los Reportes

Las VIEWS no solo muestran datos; limpian la información. Utilizamos COALESCE y NULLIF para asegurar que el Reporte de Inventario Crítico sea funcional desde el día uno, incluso para productos nuevos con "cero ventas", evitando que el sistema colapse por errores aritméticos.